

Playbook & Tutorial Técnico: Módulo de Impressão QZ Tray

Guia de Arquitetura, Autenticação RSA/X.509 e Checklist de Replicação para Outros Projetos

VERSÃO 2.0

SIGNED PRINTING RSA-2048

ESC/POS THERMAL

TANSTACK / SUPABASE / NODE

1. Visão Geral e Arquitetura

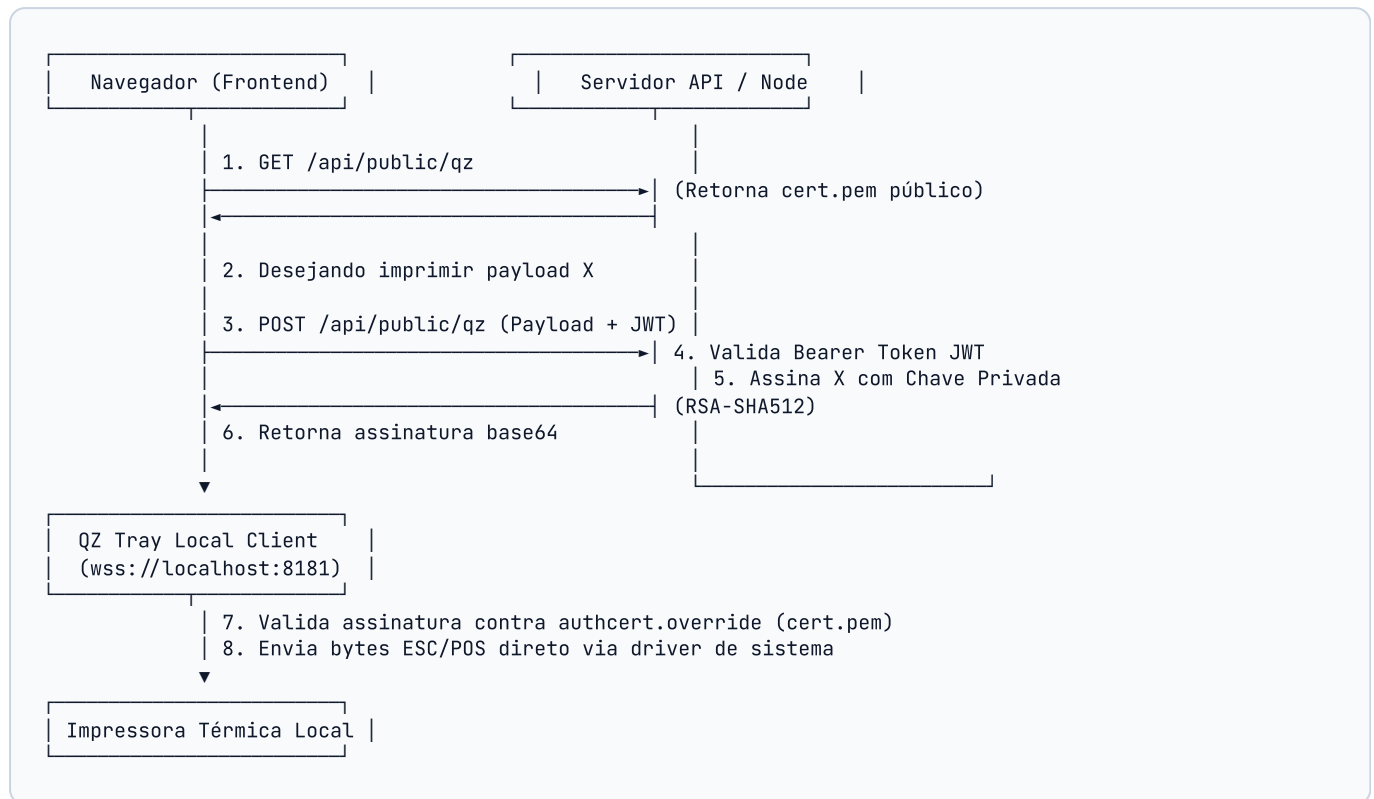
Em aplicações web de gestão (PDVs, restaurantes, ERPs), enviar impressões térmicas diretamente para impressoras locais sem exibir a janela nativa do navegador (`window.print()`) é fundamental para a produtividade operacional.

O **QZ Tray** (<https://qz.io>) é um serviço multiplataforma (Windows, macOS, Linux) que roda localmente na máquina do cliente como um daemon escutando em WebSockets seguros (`wss://localhost:8181` ou `ws://localhost:8182`).

Modo Assinado (Signed Printing) vs Modo Não Assinado:

No modo *unsigned*, o QZ Tray exibe um pop-up de permissão manual ("Allow / Deny") para o operador a cada impressão. No modo **Assinado (Signed)** implementado neste projeto, registramos um certificado digital X.509 público confiável (`cert.pem`) no QZ Tray do cliente. A cada impressão, o backend assina digitalmente o payload usando a chave privada RSA. O QZ Tray executa a impressão **100% em segundo plano, com 0 pop-ups**.

Fluxo de Comunicação e Segurança



2. Geração e Gestão do Par de Chaves Digital

Para habilitar a assinatura digital, geramos um par de chaves RSA de 2048 bits com certificado autoassinado de 10 anos via OpenSSL:

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 3650 -nodes -subj "/CN=SeuApp QZ"
```

- `cert.pem` : Certificado X.509 Público (instalado na máquina cliente).
- `key.pem` : Chave Privada RSA (armazenada estritamente nas variáveis de ambiente do servidor).

Variáveis de Ambiente (.env)

```
QZ_CERT_PEM="-----BEGIN CERTIFICATE-----\nMIIDXTCCAKWgAwIBAgIU...\n-----END CERTIFICATE-----"
QZ_PRIVATE_KEY_PEM="-----BEGIN RSA PRIVATE KEY-----\nMIIEowIBAAKCAQEAz...\n-----END RSA PRIVATE KEY-----"
```

3. Backend & Camada de Assinatura (Security Layer)

O servidor backend fornece endpoints dedicados para servimento do certificado e assinatura criptográfica dos payloads de impressão.

Endpoint Principal de Assinatura (Node / TanStack Route)

```
// src/routes/api.public.qz.ts
import { createFileRoute } from "@tanstack/react-router";
import { createSign } from "crypto";
import { getQzConfig } from "@lib/qz-config.server";

export const Route = createFileRoute("/api/public/qz")({
  server: {
    handlers: {
      GET: async () => {
        const cfg = getQzConfig();
        if (!cfg.ok) return json({ cert: "", configured: false, error: cfg.reason });
        return json({ cert: cfg.cert, configured: true, subjectCN: cfg.subjectCN });
      },
      POST: async ({ request }) => {
        // SEGURANÇA: Exige token de usuário logado para assinar payloads
        const authHeader = request.headers.get("authorization");
        if (!authHeader || !authHeader.toLowerCase().startsWith("bearer ")) {
          return json({ error: "Não autorizado." }, 401);
        }

        const cfg = getQzConfig();
        if (!cfg.ok) return json({ signature: "", configured: false, error: cfg.reason });

        const { request: toSign } = await request.json();
        const signer = createSign("RSA-SHA512");
        signer.update(toSign);
        signer.end();

        const signature = signer.sign(cfg.privateKey).toString("base64");
        return json({ signature, configured: true });
      },
    },
  },
});
```

4. Frontend & Integração Client-Side (TypeScript)

No frontend, a biblioteca `src/lib/qz-tray.ts` gerencia o ciclo de vida da conexão WebSocket, injeta os hooks de certificados do SDK e envia comandos térmicos ESC/POS.

Configuração dos Promises de Segurança

```
// src/lib/qz-tray.ts
function configureSecurity(qz: any) {
  qz.security.setSignatureAlgorithm?.("SHA512");

  // 1. Fornece o Certificado Público
  qz.security.setCertificatePromise((resolve, reject) => {
    fetch("/api/public/qz")
      .then(res => res.json())
      .then(data => data.configured ? resolve(data.cert) : reject(new Error(data.error)))
      .catch(reject);
  });

  // 2. Solicita a assinatura com o JWT do usuário logado
  qz.security.setSignaturePromise((toSign) => (resolve, reject) => {
    getAuthToken().then(token => {
      fetch("/api/public/qz", {
        method: "POST",
        headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
        body: JSON.stringify({ request: toSign })
      })
        .then(res => res.json())
        .then(data => data.configured ? resolve(data.signature) : reject(new Error(data.error)))
        .catch(reject);
    });
  });
}
```

Envio de Cupom ESC/POS e Corte de Papel

```
export function getCutSequence(cutType?: "none" | "partial" | "full"): string {
  if (cutType === "full") return "\x1DV0";
  if (cutType === "partial") return "\x1DV1";
  return "";
}

export async function printQzReceipt(printerName: string, textPayload: string, opts?: { feedLines?: number; cutType?: "none" | "partial" | "full" }) {
  const qz = await ensureQzConnected();
  const feed = Math.max(0, opts?.feedLines ?? 3);
  const cut = getCutSequence(opts?.cutType ?? "full");

  const finalPayload = textPayload + "\n".repeat(feed) + cut;
  const config = qz.configs.create(printerName, { encoding: "CP860" });
  await qz.print(config, [finalPayload]);
}
```

5. Auto-Configurador e Instalação na Máquina Cliente

Para que a máquina cliente execute impressões sem popups, o certificado `cert.pem` deve ser registrado no arquivo `qz-tray.properties` da máquina local.

Script Automático Windows (menuzin-qz-setup.bat):

O script batch automatiza a cópia do `cert.pem` para `%APPDATA%\qz\cert.pem`, acrescenta a regra `authcert.override=cert.pem` no `qz-tray.properties` e reinicia o serviço do QZ Tray automaticamente.

```
@echo off
chcp 65001 >nul
title Configuração do Módulo de Impressão QZ Tray

net session >nul 2>&1
if %errorlevel% neq 0 (
    echo [ERROR] Execute este script como ADMINISTRADOR!
    pause
    exit /b 1
)

set "QZ_DIR=%APPDATA%\qz"
if not exist "%QZ_DIR%" mkdir "%QZ_DIR%"

echo Copiando certificado...
copy /Y "%~dp0cert.pem" "%QZ_DIR%\cert.pem" >nul

echo Atualizando qz-tray.properties...
findstr /C:"authcert.override=" "%QZ_DIR%\qz-tray.properties" >nul 2>&1
if %errorlevel% neq 0 (
    echo authcert.override=cert.pem>"%QZ_DIR%\qz-tray.properties"
)

echo Reiniciando QZ Tray...
taskkill /IM "qz-tray.exe" /F >nul 2>&1
start "" "%ProgramFiles%\QZ Tray\qz-tray.exe"
echo [SUCESSO] QZ Tray configurado com sucesso!
pause
```

6. Guia de Replicação Passo a Passo em Novos Projetos

- Passo 1:** Gerar o par de chaves `cert.pem` e `key.pem` via OpenSSL.
- Passo 2:** Cadastrar `QZ_CERT_PEM` e `QZ_PRIVATE_KEY_PEM` nas variáveis de ambiente do novo projeto.
- Passo 3:** Copiar o módulo de validação (`qz-config.server.ts`) e as rotas de API (`/api/public/qz`).
- Passo 4:** Copiar a biblioteca cliente TypeScript (`qz-tray.ts`).
- Passo 5:** Disponibilizar o download do `cert.pem` e do script `setup.bat` na área de configurações da aplicação.
- Passo 6:** Invocação simples via `printQzReceipt("NOME_DA_IMPRESSORA", textoCupom)`.

7. Matriz de Diagnóstico e Resolução de Problemas (Troubleshooting)

Sintoma / Erro	Causa Provável	Solução
<code>QzNotRunningError</code>	QZ Tray não está aberto no computador do cliente.	Abrir o aplicativo QZ Tray na bandeja do Windows e verificar a porta 8181/8182.
Pop-up "Allow / Deny" aparece a cada impressão	Certificado não registrado em <code>qz-tray.properties</code> .	Executar o script <code>menuzin-qz-setup.bat</code> como Administrador na máquina cliente.
<code>401 Unauthorized</code>	Sessão expirada ou sem token Bearer.	Garantir que a chamada POST envie o cabeçalho <code>Authorization: Bearer <token></code> .
Par cert/chave não correspondem	Incompatibilidade entre as variáveis de ambiente.	Regenerar o par via OpenSSL e atualizar ambas as variáveis no servidor simultaneamente.
Caracteres acentuados corrompidos	Encoding incorreto enviado no payload.	Utilizar a opção <code>{ encoding: "CP860" }</code> na criação da config no QZ Tray.